Full Stack Developer Technical Interview Q&A

1. What is the difference between frontend and backend development?

Frontend development focuses on the client-side of applications - what users see and interact with. It involves HTML, CSS, JavaScript, and frameworks like React. Backend development handles server-side logic, databases, APIs, and business logic that users don't directly see. Full stack developers work on both sides, understanding how data flows from the database through APIs to the user interface and back. This holistic understanding allows for better architecture decisions and more efficient development.

2. Explain the Model-View-Controller (MVC) architecture pattern.

MVC separates an application into three interconnected components: Model (data and business logic), View (user interface), and Controller (handles user input and coordinates between Model and View). The Model manages data and rules, the View displays data to users, and the Controller processes user input and updates Model/View accordingly. This separation improves maintainability, testability, and allows multiple developers to work on different components simultaneously. Many frameworks like Express.js, Django, and Rails follow MVC principles.

3. What is REST and what are RESTful principles?

REST (Representational State Transfer) is an architectural style for designing web services. RESTful principles include: using standard HTTP methods (GET, POST, PUT, DELETE), stateless communication (each request contains all information needed), resource-based URLs (nouns, not verbs), using proper HTTP status codes, supporting multiple data formats (JSON, XML), and having a uniform interface. RESTful APIs are simple, scalable, and cacheable, making them the standard for web API design.

4. Explain the difference between SQL and NoSQL databases.

SQL databases are relational, use structured schemas, and store data in tables with relationships. They're ACID compliant, support complex queries with JOINs, and are ideal for structured data with clear relationships. NoSQL databases are non-relational, schema-less or

flexible schema, and can be document-based (MongoDB), key-value (Redis), column-family (Cassandra), or graph-based (Neo4j). They're better for unstructured data, horizontal scaling, and rapid development. The choice depends on data structure, scalability needs, and consistency requirements.

5. What is an API and how does it work?

An API (Application Programming Interface) is a set of protocols and tools for building software applications. It defines how different software components should interact. Web APIs allow applications to communicate over HTTP, typically using JSON or XML. When a client makes a request to an API endpoint, the server processes it, interacts with databases or other services, and returns a response. APIs enable separation of concerns, allowing frontend and backend to be developed independently and enabling third-party integrations.

6. Explain authentication vs authorization.

Authentication verifies who a user is (login process), typically using credentials like username/password, tokens, or biometrics. Authorization determines what an authenticated user is allowed to do (permissions and access control). Authentication answers "Who are you?" while authorization answers "What can you do?" Common authentication methods include JWT tokens, OAuth, session-based auth, and API keys. Authorization is often implemented using role-based access control (RBAC) or attribute-based access control (ABAC).

7. What is JWT and how does it work?

JWT (JSON Web Token) is a compact, URL-safe token format for securely transmitting information between parties. A JWT consists of three parts: header (algorithm and token type), payload (claims/data), and signature (verification). When a user logs in, the server creates a JWT and sends it to the client. The client includes this token in subsequent requests. The server verifies the signature to ensure the token hasn't been tampered with. JWTs are stateless, making them scalable, but they require careful handling of expiration and security.

8. Explain database indexing and why it's important.

Database indexing creates a data structure that improves the speed of data retrieval operations. An index is like a book's index - instead of scanning every page, you can quickly find what you need. Indexes are created on columns frequently used in WHERE clauses, JOINs, or ORDER BY statements. While indexes speed up SELECT queries, they slow down INSERT, UPDATE, and DELETE operations because the index must be maintained. Proper indexing is crucial for database performance, especially as data grows.

9. What is caching and what are common caching strategies?

Caching stores frequently accessed data in fast storage (memory) to reduce database queries and improve performance. Common strategies include: browser caching (static assets), CDN caching (content delivery networks), application-level caching (Redis, Memcached), database query caching, and HTTP caching headers. Cache invalidation strategies include time-based expiration (TTL), cache-aside pattern, write-through, and write-behind. Effective caching can dramatically improve application performance and reduce server load.

10. Explain the difference between synchronous and asynchronous programming.

Synchronous code executes sequentially - each operation waits for the previous one to complete. Asynchronous code allows operations to run concurrently without blocking. In JavaScript, callbacks, Promises, and async/await enable asynchronous programming. Asynchronous operations are essential for I/O-bound tasks like API calls, file operations, and database queries, as they prevent the application from freezing while waiting for responses. This is crucial for building responsive applications that can handle multiple requests simultaneously.

11. What is Docker and what are its benefits?

Docker is a containerization platform that packages applications and their dependencies into containers - lightweight, portable, and consistent environments. Benefits include: consistency across development, staging, and production; isolation between applications; easy scaling; simplified deployment; and resource efficiency compared to virtual machines. Docker containers run on any system with Docker installed, eliminating "it works on my machine" problems. Docker Compose allows managing multi-container applications easily.

12. Explain microservices architecture vs monolithic architecture.

A monolithic architecture is a single, unified application where all components are tightly coupled and deployed together. Microservices architecture breaks applications into small, independent services that communicate via APIs. Monoliths are simpler to develop and deploy initially but become harder to scale and maintain as they grow. Microservices offer better scalability, technology diversity, and fault isolation, but add complexity in deployment, testing, and service communication. The choice depends on team size, application complexity, and scalability needs.

13. What is CI/CD and why is it important?

CI/CD (Continuous Integration/Continuous Deployment) automates the software delivery process. CI automatically builds and tests code when changes are pushed, catching issues early. CD automatically deploys code to production after passing tests. Benefits include: faster releases, reduced manual errors, consistent deployments, early bug detection, and the ability to roll back quickly. Common tools include GitHub Actions, Jenkins, GitLab CI, and CircleCI. CI/CD is essential for modern software development, enabling rapid iteration and reliable deployments.

14. Explain database normalization and denormalization.

Normalization organizes data to reduce redundancy and improve data integrity. It involves splitting data into related tables and eliminating duplicate data. Normal forms (1NF, 2NF, 3NF) define rules for this process. Denormalization intentionally introduces redundancy to improve query performance, often by duplicating data or combining tables. While normalization reduces storage and maintains consistency, it can require more JOINs. Denormalization speeds up reads but requires careful management of data consistency. The choice depends on read vs write patterns.

15. What is load balancing and how does it work?

Load balancing distributes incoming network traffic across multiple servers to ensure no single server is overwhelmed. It improves availability, reliability, and performance. Common algorithms include round-robin (distribute sequentially), least connections (send to server with fewest active connections), and IP hash (route based on client IP). Load balancers can be hardware-based or software-based (like Nginx, HAProxy). They also provide health checks to remove unhealthy servers from the pool and enable horizontal scaling.

16. Explain the CAP theorem.

The CAP theorem states that in a distributed system, you can only guarantee two out of three properties: Consistency (all nodes see the same data simultaneously), Availability (system remains operational), and Partition tolerance (system continues despite network failures). Since partition tolerance is unavoidable in distributed systems, you typically choose between CP (consistency and partition tolerance) or AP (availability and partition tolerance). Understanding CAP helps in choosing appropriate databases and system architectures for your use case.

17. What is GraphQL and when should you use it?

GraphQL is a query language and runtime for APIs that allows clients to request exactly the data they need. Unlike REST, which has fixed endpoints, GraphQL has a single endpoint where clients specify their data requirements. Benefits include: reduced over-fetching and under-fetching, strong typing, introspection capabilities, and efficient data loading. It's ideal when you have multiple clients with different data needs, complex data relationships, or when you want to reduce API versioning. However, it adds complexity and may not be necessary for simple CRUD operations.

18. Explain the difference between SQL injection and how to prevent it.

SQL injection is a security vulnerability where attackers inject malicious SQL code into input fields, potentially accessing or modifying the database. Prevention methods include: using parameterized queries/prepared statements (never concatenate user input into SQL), input validation and sanitization, using ORMs that handle escaping automatically, implementing least privilege database access, and using stored procedures. Parameterized queries ensure user input is treated as data, not executable code, which is the most effective prevention method.

19. What is Redis and what are its use cases?

Redis is an in-memory data structure store used as a database, cache, and message broker. It's extremely fast because data is stored in memory. Common use cases include: caching frequently accessed data, session storage, real-time analytics, message queues, rate limiting, leaderboards, and pub/sub messaging. Redis supports various data structures (strings, hashes, lists, sets, sorted sets) and provides persistence options. It's essential for high-performance applications requiring low-latency data access.

20. Explain the difference between horizontal and vertical scaling.

Vertical scaling (scaling up) adds more power to existing servers (more CPU, RAM, storage). It's simpler but has physical and cost limits. Horizontal scaling (scaling out) adds more servers to handle increased load. It's more flexible and cost-effective at scale but requires load balancing and distributed system considerations. Modern applications typically use horizontal scaling for better scalability and availability. Cloud platforms make horizontal scaling easier with auto-scaling features.

21. What is WebSocket and when would you use it?

WebSocket provides a persistent, bidirectional communication channel between client and server over a single TCP connection. Unlike HTTP's request-response model, WebSockets allow real-time, two-way communication. Use cases include: chat applications, live notifications, real-time collaboration tools, live data feeds (stock prices, sports scores), online gaming, and IoT device communication. WebSockets reduce overhead compared to polling and provide lower latency for real-time applications. Libraries like Socket.io simplify WebSocket implementation.

22. Explain the difference between PUT and PATCH HTTP methods.

PUT is idempotent and replaces the entire resource with the provided data. If you PUT a resource, you're sending the complete representation. PATCH is used for partial updates - you only send the fields you want to change. PATCH is not necessarily idempotent. Use PUT when you're replacing the entire resource, and PATCH when you're updating specific fields. This distinction helps maintain RESTful API design principles and provides clear semantics for API consumers.

23. What is OAuth and how does it work?

OAuth is an authorization framework that allows third-party services to access user resources without sharing passwords. The OAuth flow typically involves: user requests access, application redirects to authorization server, user authenticates and grants permission, authorization server returns an authorization code, application exchanges code for access token, and application uses token to access protected resources. OAuth 2.0 is the current standard, used by services like Google, GitHub, and Facebook for "Sign in with X" functionality. It provides secure, delegated access to resources.

24. Explain database transactions and ACID properties.

A transaction is a sequence of database operations that must execute completely or not at all (all-or-nothing). ACID properties ensure reliability: Atomicity (all operations succeed or all fail), Consistency (database remains in valid state), Isolation (concurrent transactions don't interfere), and Durability (committed changes persist even after system failure). Transactions are crucial for maintaining data integrity, especially in financial systems where partial updates could cause serious problems. Database systems use locking and logging to implement ACID properties.

25. What is API rate limiting and why is it important?

Rate limiting restricts the number of requests a client can make within a time period. It prevents abuse, protects against DDoS attacks, ensures fair resource usage, and helps maintain system stability. Common strategies include: fixed window (X requests per minute), sliding window (more accurate), token bucket (allows bursts), and leaky bucket (smooth rate). Rate limiting can be implemented at the API gateway, application level, or using services like Redis. Proper rate limiting is essential for production APIs.

26. Explain the difference between stateless and stateful applications.

Stateless applications don't store client state between requests - each request contains all information needed. Stateful applications maintain client state on the server between requests. Stateless applications are more scalable, work better with load balancers, and are easier to scale horizontally. Stateful applications can provide better user experience for complex workflows but require session management and can be harder to scale. Modern web

applications typically use stateless APIs with client-side state management, while maintaining server-side state only when necessary (like sessions).

27. What is message queuing and when would you use it?

Message queuing is an asynchronous communication pattern where messages are stored in a queue until processed. Producers send messages to queues, and consumers process them. Use cases include: decoupling services, handling peak loads, ensuring reliable delivery, implementing event-driven architectures, and processing long-running tasks asynchronously. Popular message queue systems include RabbitMQ, Apache Kafka, AWS SQS, and Redis. Message queues enable scalable, resilient systems by allowing services to communicate asynchronously and handle failures gracefully.

28. Explain the difference between HTTP and HTTPS.

HTTP (Hypertext Transfer Protocol) is the protocol for transferring data over the web. HTTPS is HTTP secured with SSL/TLS encryption. HTTPS encrypts data in transit, preventing interception and tampering. It also provides authentication through certificates, ensuring you're communicating with the intended server. HTTPS is now standard for all web applications, required for features like service workers, and improves SEO rankings. Browsers mark HTTP sites as "not secure," making HTTPS essential for user trust and security.

29. What is database sharding and how does it work?

Sharding is a database partitioning technique that splits a large database into smaller, more manageable pieces called shards. Each shard is a separate database containing a subset of data. Sharding is used for horizontal scaling when a single database becomes too large or slow. Common sharding strategies include: range-based (split by value ranges), hash-based (distribute by hash function), and directory-based (lookup table for shard location). While sharding improves performance and scalability, it adds complexity in queries spanning multiple shards and data rebalancing.

30. Explain the importance of logging and monitoring in production systems.

Logging records application events and errors for debugging and auditing. Monitoring tracks system health, performance metrics, and alerts on issues. Both are crucial for: identifying and diagnosing problems quickly, understanding system behavior, tracking user activity, meeting compliance requirements, and making data-driven decisions. Effective logging includes structured logs, appropriate log levels, and centralized log aggregation. Monitoring should track key metrics (CPU, memory, response times, error rates) and set up alerts for anomalies. Tools like ELK stack, Prometheus, Grafana, and Datadog help with logging and monitoring.